

University
Politehnica
of Bucharest





PL/SQL (IV)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro



Overview

- PL/SQL Error Handling
- Cursors



Overview

- PL/SQL Error Handling
- Cursors



PL/SQL Error Handling

- Exceptions (PL/SQL runtime errors) can arise from
  - Design faults
  - Coding mistakes
  - Hardware failures
  - Many other sources

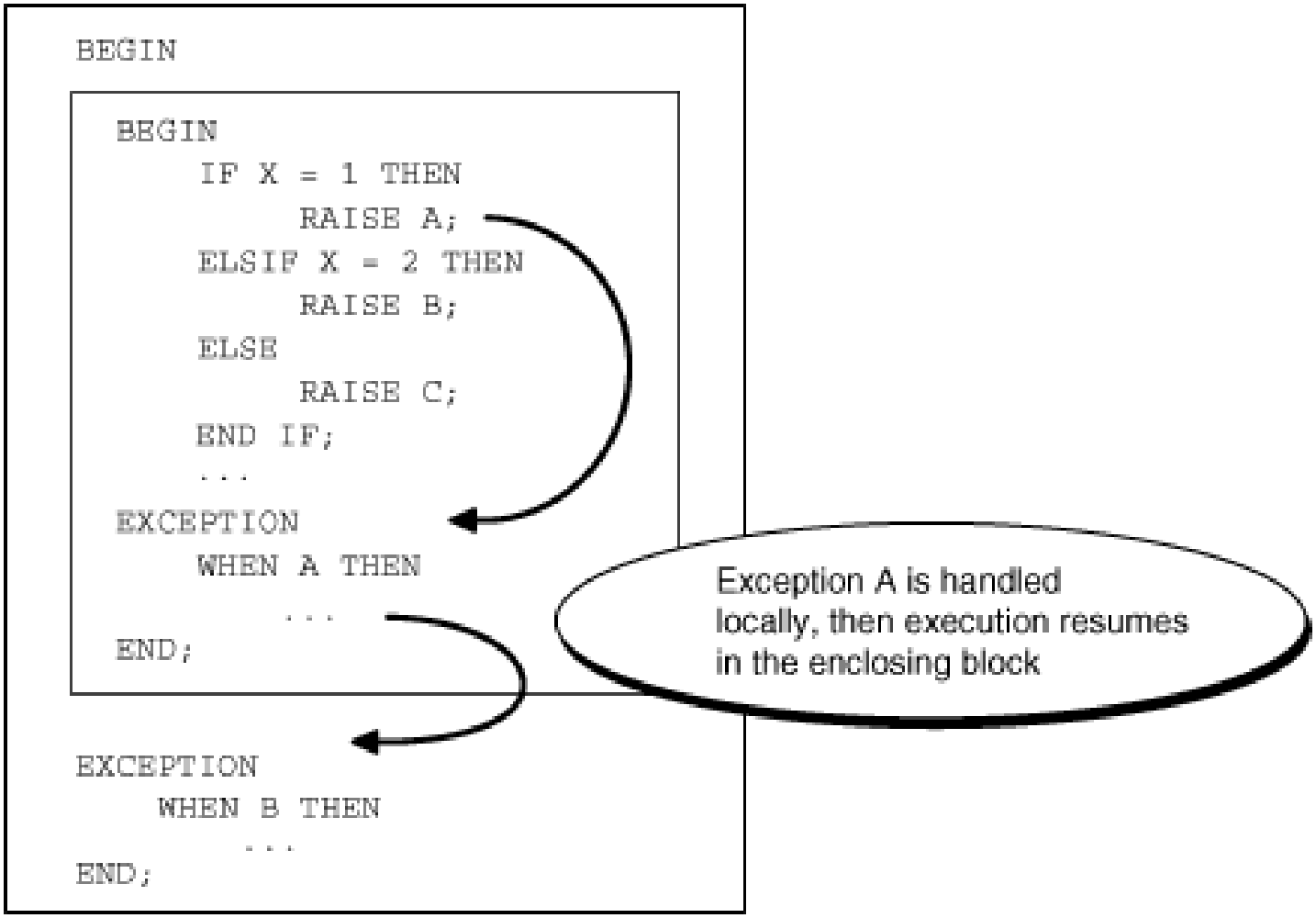
- A developer
  - Cannot anticipate all possible exceptions
  - Can write exception handlers that lets the program continue to operate in their presence

- Any PL/SQL block can have an exception-handling part:

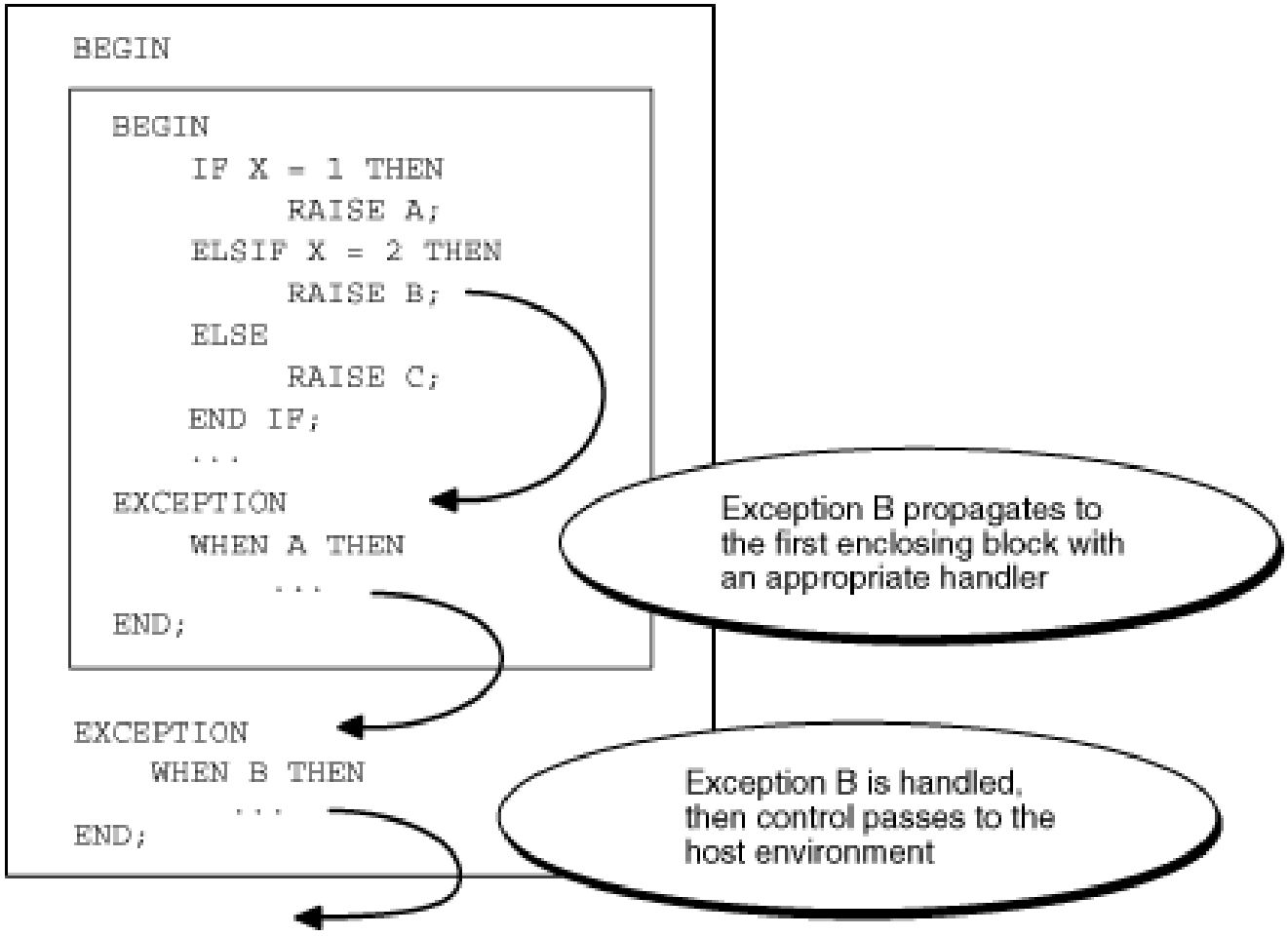
EXCEPTION

- The exception-handling part can have one or more exception handlers

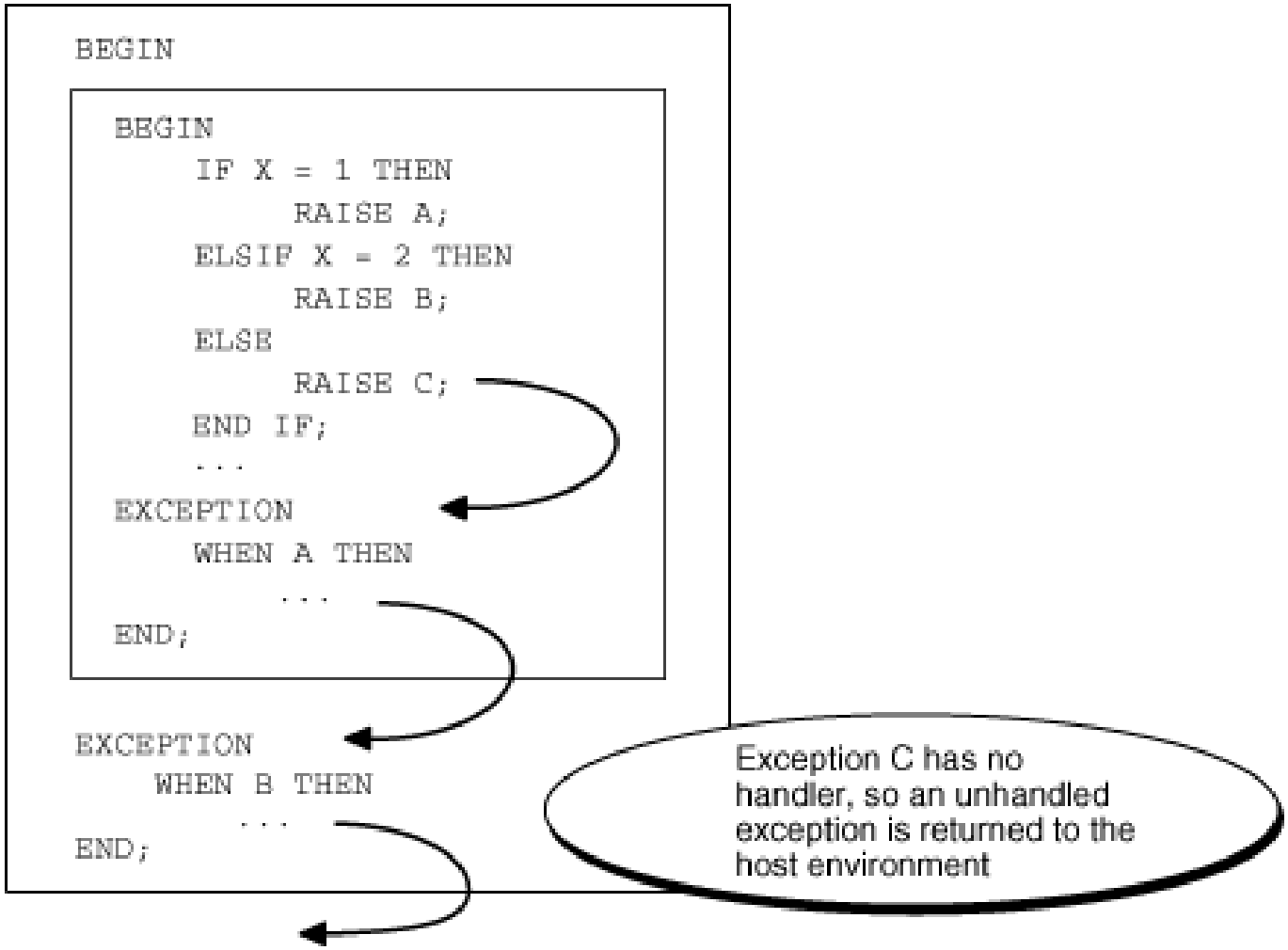
PL/SQL Error Propagation



PL/SQL Error Propagation



PL/SQL Error Propagation





PL/SQL Error Propagation

- When an exception is raised in the executable part of the block
  - The executable part stops
  - Control transfers to the exception-handling part

- If there is no enclosing block:
  - If the exception handler is in a subprogram, then control returns to the invoker, at the statement after the invocation.
  - If the exception handler is in an anonymous block, then control transfers to the host environment (for example, SQL*Plus)

- If there is an enclosing block:
  - Control transfers to the next statement of the enclosing block after an exception handler runs
  - The exception propagates if an exception is raised in a block that has no exception handler
  - The exception reproduces itself in successive enclosing blocks until a block has a handler for it or there is no enclosing block
  - If there is no handler for the exception, then PL/SQL returns an unhandled exception error to the invoker or host environment, which determines the outcome

PL/SQL Error Handling

- Exception categories:
  1. Internally defined
     - The runtime system raises internally defined exceptions implicitly (automatically).
     - Always has an error code
     - Does not have a name unless PL/SQL gives it one or you give it one
     - E.g.
       - ORA-00060 (deadlock detected while waiting for resource)
       - ORA-27102 (out of memory)

PL/SQL Error Handling

- The exception categories are:
  2. Predefined
     - A predefined exception is an internally defined exception that PL/SQL has given a name.
     - E.g.: ORA-06500 (PL/SQL: storage error) has the predefined name STORAGE_ERROR

  3. User-defined
     - Developers can declare their own exceptions in the declarative part of any PL/SQL
       - Anonymous block
       - Subprogram
       - Package
     - User-defined exceptions are raised explicitly
     - E.g.: Declare an exception named insufficient_funds to flag overdrawn bank accounts



PL/SQL Error Handling

Category	Definer	Has Error Code	Has Name	Raised Implicitly	Raised Explicitly
Internally defined	Runtime system	Always	Only if you assign one	Yes	Optionally
Predefined	Runtime system	Always	Always	Yes	Optionally
User-defined	User	Only if you assign one	Always	No	Always

- SQLCODE – function to get the error code
- SQLERRM – function to get the error message



PL/SQL Error Handling
Internally defined exception

- Internally defined exceptions (ORA-n errors)
  - Are described in Oracle Database Error Messages Reference
  - The runtime system raises them implicitly (automatically).
  - Does not have a name unless either PL/SQL gives it one or a developer gives it one
  - Can be handled only with **OTHERS** exception handlers.

- To give a name to an internally defined exception, do the following in the declarative part of the appropriate anonymous block, subprogram, or package:
  - Declare the name
  - Associate the name with the error code of the internally defined exception



PL/SQL Error Handling
Internally defined exception example

```
INSERT INTO employees(first_name, last_name)
VALUES('Ion', 'Ionescu');
```



PL/SQL Error Handling
Internally defined exception example

```
 1 DECLARE
 2   insert_null_exp exception;
 3   PRAGMA exception_init(insert_null_exp, -1400);
 4 BEGIN
 5   INSERT INTO employees(first_name, last_name)
 6   VALUES('Ion', 'Ionescu');
 7 EXCEPTION
 8   WHEN insert_null_exp THEN
 9     dbms_output.put_line('Error msg  '||SQLERRM);
10     dbms_output.put_line('Error code '||SQLCODE);
11   WHEN others THEN
12     dbms_output.put_line('Others error');
13 END;
14 /
```



PL/SQL Error Handling Pragma

- A pragma is an instruction to the compiler that is processed at compilation time
- A pragma begins with the reserved word PRAGMA followed by the name of the pragma
- Some pragmas have arguments
- A pragma may appear before a declaration or a statement
- Additional restrictions may apply for specific pragmas
- The extent of a pragma's effect depends on the pragma
- A pragma whose name or argument is not recognized by the compiler has no effect.



PL/SQL Error Handling
Predefined exceptions

- Predefined exceptions are internally defined exceptions that have predefined names
- PL/SQL declares the names of the predefined exceptions globally in the package STANDARD
- The runtime system raises predefined exceptions implicitly (automatically)
- Developers can write exception handlers specifically for predefined exceptions because they have names
- To explicitly raise an exception use RAISE



PL/SQL Error Handling
Predefined exceptions example

```
 1 DECLARE
 2   n number := &n;
 3 BEGIN
 4   IF n < 0 THEN
 5     RAISE INVALID_NUMBER;   -- raise explicitly
 6   END IF;
 7 EXCEPTION
 8   WHEN INVALID_NUMBER THEN
 9     dbms_output.put_line('n < 0');
10     dbms_output.put_line('Error msg  '||SQLERRM);
11     dbms_output.put_line('Error code '||SQLCODE);
12   WHEN others THEN
13     dbms_output.put_line('Others error');
14 END;
15 /
```



PL/SQL Error Handling
User-defined exceptions

- Developers can declare their own exceptions in the declarative part of any PL/SQL anonymous block, Subprogram, or Package

- An exception name declaration has this syntax:

exception_name exception;

- Developers can attach to a user defined exception
  - An exception code (-20999 și -20000) using

PRAGMA exception_init(exception_name, exception_code);

  - An exception message using the procedure

RAISE_APPLICATION_ERROR(exception_code, exception_message [, TRUE | FALSE]);

  - **TRUE** – PL/SQL puts **exception_code** on top of the error stack
  - **FALSE (default) –** PL/SQL replaces the error stack with **exception_code** (i.e., all previous errors are replaced with the current one)



PL/SQL Error Handling
User-defined exceptions Example

```
DECLARE
  n number := &n;
  my_exp1 exception;
  my_exp2 exception;
  PRAGMA exception_init(my_exp1, -20100);
  PRAGMA exception_init(my_exp2, -20200);
BEGIN
  IF n >= 200  THEN
    RAISE my_exp1;
  ELSIF n >= 100 THEN
    raise_application_error(-20200, 'Numar > 200');
  END IF;
EXCEPTION
  WHEN my_exp1 THEN
    dbms_output.put_line('n >100');
    dbms_output.put_line('my_exp1 error msg  '||SQLERRM);
    dbms_output.put_line('my_exp1 error code '||SQLCODE);
   WHEN my_exp2 THEN
    dbms_output.put_line('n > 200');
    dbms_output.put_line('my_exp2 error msg  '||SQLERRM);
    dbms_output.put_line('my_exp2 error code '||SQLCODE);
  WHEN others THEN
    dbms_output.put_line('Others error');
END;
/
```



Overview

- PL/SQL Error Handling
- Cursors



PL/SQL Static SQL

- Static SQL is a PL/SQL feature that allows SQL syntax to be used directly in a block

- PL/SQL static SQL statements (some have a modified syntax):
  - SELECT
  - INSERT
  - UPDATE
  - DELETE
  - MERGE
  - COMMIT
  - ROLLBACK
  - SAVEPOINT
  - SET TRANSACTION
  - LOCK TABLE



PL/SQL Static SQL

- A PL/SQL static SQL statement can have a PL/SQL identifier (names given to a PL/SQL object) wherever its SQL counterpart can have a placeholder for a bind variable

- The PL/SQL identifier must identify either a variable or a formal parameter

- Bind variables
  - Are used to reuse the query parse tree and execution plan when new values are given
  - Can be used in dynamic PL/SQL code, but that's not good practice!
  - Are a placeholder for values within the query.
  - Unlike substitution variables, these are not prompted for when you come to compile the code.

PL/SQL Dynamic SQL

```
DECLARE
    v_sql varchar2(200);
    v_dname departments.department_name%type;
    id_dept departments.department_id%type := &id;
BEGIN
    v_sql := 'SELECT department_name FROM departments WHERE department_id = :1';
    EXECUTE IMMEDIATE v_sql INTO v_dname USING id_dept;
    dbms_output.put_line(v_dname);
END;
/
```



PL/SQL Static SQL

```
 1 DECLARE
 2     v_sql varchar2(200);
 3     v_dname departments.department_name%type;
 4     id_dept departments.department_id%type := &id;
 5 BEGIN
 6     SELECT department_name
 7     INTO v_dname
 8     FROM departments
 9     WHERE department_id = id_dept;
10
11     dbms_output.put_line(v_dname);
12 END;
13 /
14
15 -- SQL Engine:
16 SELECT department_name FROM departments WHERE department_id = :1;
```



PL/SQL Static SQL

- The values of some variables are undefined after PL/SQL code runs a DML statement:
  - After a FETCH or SELECT statement raises an exception
    - The values of the define variables after that statement are undefined.
  - After a DML statement that affects zero rows
    - The values of the OUT bind variables are undefined unless the DML statement is a BULK or multiple-row operation



PL/SQL Static SQL

- Static SQL includes these SQL pseudocolumns (for different types of objects):
  - CURRVAL and NEXTVAL
  - LEVEL
    - Hierarchical Query Pseudocolumns (SELECT queries with START WITH ... CONNECT BY)
    - Returns 1 for a root row, 2 for a child of a root, and so on.
  - OBJECT_VALUE
    - Returns system-generated names for the columns of an object table, XMLType table, object view, or XMLType view
    - Useful for
      - Identifying the value of a substitutable row in an object table
      - Creating object views with the WITH OBJECT IDENTIFIER clause.
  - ROWID - returns the physical address of the row
  - ROWNUM - returns a number indicating the order in which Oracle selects the row from a table or set of joined rows for each row returned by a query



PL/SQL Static SQL

- To access the values of a sequence in SQL statements:
  - CURRVAL pseudocolumn
    - Returns the current value of the sequence
  - NEXTVAL pseudocolumn
    - Increments the sequence
    - Returns the new value

- Use dot notation to reference these pseudocolumns :

  sequence_name.CURRVAL

  sequence_name.NEXTVAL

- Each time sequence_name.NEXTVAL is reference
  - The sequence is incremented immediately and permanently
  - Commit or roll back on the transaction will not decrement the sequence



PL/SQL Static SQL

- Use sequence_name.CURRVAL and sequence_name.NEXTVAL in a PL/SQL expression wherever NUMBER expression can be used.

- Note:
  - Using sequence_name.CURRVAL or sequence_name.NEXTVAL to provide a **default value for an ADT method** parameter causes a **compilation error**
  - **PL/SQL evaluates every occurrence** of sequence_name.CURRVAL and sequence_name.NEXTVAL
  - **SQL evaluates a sequence expression for every row** in which it appears.



Cursors

- A cursor is a pointer to a private SQL area that stores information about processing a specific SELECT or DML statement
- A session cursor lives in session memory until the session ends
- When the session ends the cursor ceases to exist
- Cursor types:
  - Implicit
    - Constructed and managed by PL/SQL
  - Explicit
    - Construct and manage by the developer



Cursors

- Cursor attributes
  - Are used to get information about any session cursor
  - Can be referenced in procedural statements, but not in SQL statements
- Query the dynamic performance view V$OPEN_CURSOR to list the session cursors that each user session currently has opened and parsed
- The number of cursors that a session can have open simultaneously is determined by:
  - The amount of memory available to the session
  - The value of the initialization parameter OPEN_CURSORS



Cursors

- Generally, PL/SQL parses
  - An explicit cursor only the first time the session opens it
  - A SQL statement (creating an implicit cursor) only the first time the statement runs
- All parsed SQL statements are cached
- A SQL statement is reparsed only if it is aged out of the cache by a new SQL statement
- PL/SQL does not reparse the associated query if an explicit cursor is closed and immediately reopened



Implicit Cursors

- An implicit cursor is a session cursor that is constructed and managed by PL/SQL
- PL/SQL opens an implicit cursor every time SELECT or DML statement is executed
- A developer:
  - Cannot control an implicit cursor
  - Can get information from its attributes.



Implicit Cursors

- The syntax of an implicit cursor attribute value is SQL%attribute
- An implicit cursor is also called a SQL cursor
- SQL%attribute always refers to the most recently run SELECT or DML statement
- If no such statement has run, the value of SQL%attribute is NULL
- An implicit cursor closes after its associated statement runs
- Its attribute values remain available until another SELECT or DML statement runs



Implicit Cursors

- The most recently run SELECT or DML statement might be in a different scope:
  - To save an attribute value for later use, assign it to a local variable immediately
  - Otherwise, other operations, such as subprogram invocations, might change the value of the attribute before testing it
- The implicit cursor attributes are:
  - SQL%ISOPEN Attribute: Is the Cursor Open?
  - SQL%FOUND Attribute: Were Any Rows Affected?
  - SQL%NOTFOUND Attribute: Were No Rows Affected?
  - SQL%ROWCOUNT Attribute: How Many Rows Were Affected?
  - SQL%BULK_ROWCOUNT
  - SQL%BULK_EXCEPTIONS



Implicit Cursors

- SQL%ISOPEN always returns FALSE
  - An implicit cursor always closes after its associated statement runs
- SQL%FOUND returns:
  - NULL if no SELECT or DML statement has run
  - TRUE if a SELECT statement returned one or more rows or a DML statement affected one or more rows
  - FALSE otherwise
- SQL%NOTFOUND returns:
  - NULL if no SELECT or DML statement has run
  - FALSE if a SELECT statement returned one or more rows or a DML statement affected one or more rows
  - TRUE otherwise



Implicit Cursors

- The SQL%NOTFOUND attribute is not useful with the PL/SQL SELECT INTO statement:
  - If the SELECT INTO statement returns no rows, PL/SQL raises the predefined exception NO_DATA_FOUND immediately, before you can check SQL%NOTFOUND.
  - A SELECT INTO statement that invokes a SQL aggregate function always returns a value (possibly NULL), thus the SQL%NOTFOUND attribute is always FALSE, so checking it is unnecessary.



Implicit Cursors

- SQL%ROWCOUNT returns:
  - NULL if no SELECT or DML statement has run
  - Otherwise, the number of rows returned by a SELECT statement or affected by a DML statement (an INTEGER)
- If a SELECT INTO statement without a BULK COLLECT clause returns multiple rows
  - PL/SQL raises the predefined exception TOO_MANY_ROWS and SQL%ROWCOUNT returns 1
  - This is not the actual number of rows that satisfy the query
- The value of SQL%ROWCOUNT attribute is unrelated to the state of a transaction
- The value of SQL%ROWCOUNT is not restored
  - To the value it had before the savepoint when a transaction rolls back to a savepoint
  - To the original value in the parent transaction when an autonomous transaction ends



Implicit Cursors

- To allow a FORALL statement to continue even if some of its DML statements fail, include the SAVE EXCEPTIONS clause
- Use SQL%BULK_EXCEPTIONS in the FORALL clause to find the element that caused an error
- Use SQL%BULK_ROWCOUNT to get the total number of rows affected by the FORALL statement



Implicit Cursors

```
 1 set serveroutput on
 2 DECLARE
 3   TYPE rec IS RECORD(
 4     country_id countries.country_id%TYPE,
 5     country_name countries.country_name%TYPE,
 6     region_id countries.region_id%TYPE
 7   );
 8   TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;
 9   v aa; -- create a null associative array
10   err_idx PLS_INTEGER;
11   err_count PLS_INTEGER;
12 BEGIN
13
14   v := aa(1 => rec('RO', 'Romania', 1), 2 => rec('MD', 'Moldova', 1),
15           5 => rec('BG', 'Bulgaria', 1));
16
17   FORALL j IN v.FIRST .. v.LAST SAVE EXCEPTIONS
18   INSERT INTO COUNTRIES VALUES v(j);
19
20 EXCEPTION
21   WHEN others THEN
22     DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);
23     err_count := SQL%BULK_EXCEPTIONS.COUNT;
24     FOR i IN 1..err_count LOOP
25       DBMS_OUTPUT.PUT_LINE ('SQL Code '||SQL%BULK_EXCEPTIONS(i).ERROR_CODE);
26       err_idx := SQL%BULK_EXCEPTIONS(i).ERROR_INDEX;
27       DBMS_OUTPUT.PUT_LINE('Bad statement #: ' || err_idx);
28     END LOOP;
29
30     FOR i IN v.FIRST..v.LAST LOOP
31       DBMS_OUTPUT.PUT_LINE('INSERT '|| i ||' EXECUTETD? '||SQL%BULK_ROWCOUNT(i));
32     END LOOP;
33
34     ROLLBACK;
35 END;
36 /
```



Explicit Cursors

- An explicit cursor is a session cursor that the developer constructs and manages

- The developer
  - Declares and defines an explicit cursor
  - Gives the cursor name associating a query with it (typically, the query returns multiple rows).
  - Can process the query result set in either of these ways:
    - Using **OPEN-FETCH-CLOSE**
      - Open the explicit cursor (with the OPEN statement)
      - Fetch rows from the result set (with the FETCH statement)
      - Close the explicit cursor (with the CLOSE statement)
    - Use the explicit cursor in a cursor **FOR LOOP** statement



Explicit Cursors

- The developer cannot
  - Assign a value to an explicit cursor
  - Use a cursor in an expression
  - Use a cursor as a formal subprogram parameter or a host variable
- A developer can reference an explicit cursor or cursor variable by its name unlike an implicit cursor
- An explicit cursor or cursor variable is called a named cursor



Declaring and Defining Explicit Cursors

- The developer can either
  - Declare an explicit cursor first and then define it later in the same block, subprogram, or package,
  - Declare and define it at the same time.

- An explicit cursor declaration, which only declares a cursor, has this syntax:

CURSOR cursor_name [parameter_list] RETURN return_type;

- An explicit cursor definition has this syntax:

CURSOR cursor_name [parameter_list] [RETURN return_type] IS select_statement;



Opening an Explicit Cursors

- After declaring and defining an explicit cursor, the developer can open it using the OPEN statement

- The OPEN statement does the following:

1. Allocates database resources to process the query
2. Processes the query:
   - Identifies the result set
     - If the query references variables or cursor parameters, their values affect the result set
   - If the query has a FOR UPDATE clause, locks the rows of the result set
3. Positions the cursor before the first row of the result set



Fetching Data with Explicit Cursors

- After opening an explicit cursor, the developer can fetch the rows of the query result set with the FETCH statement
- The basic syntax of a FETCH statement that returns one row is:

FETCH cursor_name INTO into_clause;

- The **into_clause** is either a list of variables or a single record variable
- The variable list or record must have a corresponding type-compatible variable or field for each column that the query returns
- The %TYPE and %ROWTYPE attributes are useful for declaring variables and records for use in FETCH statements



Fetching Data with Explicit Cursors

- The FETCH statement
  - Retrieves the current row of the result set
  - Stores the column values of that row into the variables or record
  - Advances the cursor to the next row
- The developer can
  - Use the FETCH statement inside a LOOP statement
  - Exit the LOOP when the FETCH statement runs out of rows (use %NOTFOUND)
- PL/SQL does not raise an exception when a FETCH statement returns no rows



Closing an Explicit Cursors

- The developer can close an open explicit cursor with the CLOSE statement
- The CLOSE statement allows the allocated resources for the cursor to be reused.
- After closing a cursor, t**he developer cannot**
  - Fetch records from the cursor's result set
  - Reference the cursor's attributes.
  - PL/SQL raises the predefined exception INVALID_CURSOR otherwise
- The developer can reopen a closed cursor
- The explicit cursor must be closed before trying to reopen it.
  - PL/SQL raises the predefined exception CURSOR_ALREADY_OPEN otherwise



Explicit Cursors
%ISOPEN and %ROWCOUNT Attributes

- %ISOPEN returns
  - TRUE if its explicit cursor is open
  - FALSE otherwise.

- %ISOPEN is useful for checking that an explicit cursor is not already open before you try to open it

- If a developer tries to open an explicit cursor that is already open PL/SQL raises the predefined exception CURSOR_ALREADY_OPEN

- %ROWCOUNT returns:
  - Zero after the explicit cursor is opened but before the first fetch
  - Otherwise, the number of rows fetched (an INTEGER)



Explicit Cursors
%FOUND and %NOTFOUND Attributes

- %FOUND returns:
  - NULL after the explicit cursor is opened but before the first fetch
  - TRUE if the most recent fetch from the explicit cursor returned a row
  - FALSE otherwise

- %FOUND is useful for determining whether there is a fetched row to process.

- %NOTFOUND returns:
  - NULL after the explicit cursor is opened but before the first fetch
  - FALSE if the most recent fetch from the explicit cursor returned a row
  - TRUE otherwise

- %NOTFOUND is useful for exiting a loop when FETCH fails to return a row



Explicit Cursors Example

```
set serveroutput on

DECLARE
  CURSOR c1 IS
    SELECT region_name, country_name cn, country_id
    FROM regions NATURAL JOIN countries ORDER BY 1, 2;
  CURSOR c2 RETURN locations%ROWTYPE IS
    SELECT  *
    FROM locations
    ORDER BY city;
  r1 c1%ROWTYPE;
  r2 locations%ROWTYPE;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    OPEN c2;
    LOOP
      FETCH c2 INTO r2;
      EXIT WHEN c2%NOTFOUND;
      IF r2.country_id = r1.country_id THEN
        DBMS_OUTPUT.PUT_LINE(r1.region_name
                    || '::' || r1.cn ||'::'||r2.city);
      END IF;
    END LOOP;
    IF c2%ISOPEN THEN
      CLOSE c2;
    END IF;
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
END;
/
```



Variables in Explicit Cursors Queries

- An explicit cursor query can reference any variable in its scope
- When opening an explicit cursor
  - PL/SQL evaluates any variables in the query
  - Uses those values when identifying the result set
- Changing the values of the variables later does not change the result set



Explicit Cursors Example

```
 1 set serveroutput on
 2 DECLARE
 3   idr regions.region_id%TYPE := &id;
 4   CURSOR c1 IS
 5     SELECT region_name rn, country_name cn, city
 6     FROM regions NATURAL JOIN countries NATURAL JOIN LOCATIONS
 7     WHERE region_id = idr
 8     ORDER BY 1, 2;
 9   r1 c1%ROWTYPE;
10 BEGIN
11   OPEN c1;
12   LOOP
13     FETCH c1 INTO r1;
14     EXIT WHEN c1%NOTFOUND;
15     DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r1.city);
16   END LOOP;
17   IF c1%ISOPEN THEN
18     CLOSE c1;
19   END IF;
20 END;
21 /
```



Explicit Cursors Parameters

- Developers can
  - Create an explicit cursor that has formal parameters
  - Pass different actual parameters to the cursor each time you open it
- A developer can use a formal cursor parameter anywhere a constant can be used in the cursor query
- A developer cannot reference formal cursor parameters outside the cursor query
- Use different names for formal and actual cursor parameters to avoid confusion
- A developer can specify a default value for formal parameters of a cursor



Explicit Cursors Parameters Example

```
1  set serveroutput on
2  DECLARE
3    CURSOR c1(idr regions.region_id%TYPE DEFAULT 1, idc varchar) IS
4      SELECT region_name rn, country_name cn, city
5      FROM regions NATURAL JOIN countries NATURAL JOIN LOCATIONS
6      WHERE region_id = idr AND country_id = idc
7      ORDER BY 1, 2;
8    r1 c1%ROWTYPE;
9  BEGIN
10   OPEN c1(idc => 'UK');
11   LOOP
12     FETCH c1 INTO r1;
13     EXIT WHEN c1%NOTFOUND;
14     DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r1.city);
15   END LOOP;
16   IF c1%ISOPEN THEN
17     CLOSE c1;
18   END IF;
19
20   OPEN c1(idr=> 2, idc => 'US');
21   LOOP
22     FETCH c1 INTO r1;
23     EXIT WHEN c1%NOTFOUND;
24     DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r1.city);
25   END LOOP;
26   IF c1%ISOPEN THEN
27     CLOSE c1;
28   END IF;
29 END;
30 /
```



Cursors FOR LOOP Statement

- The cursor FOR LOOP statement lets developers run a SELECT statement and then immediately loop through the rows of the result set

- This statement can use either an implicit or explicit cursor (but not a cursor variable)

- If only a SELECT statement is specified inside the FOR LOOP statement
  - This form of the cursor FOR LOOP statement uses an implicit cursor
  - Is called an implicit cursor FOR LOOP statement
  - The implicit cursor is internal to the statement
  - **The statement cannot be referenced with the name SQL**

- If an explicit cursor is specified inside the FOR LOOP statement
  - Is called an explicit cursor FOR LOOP statement



Cursors FOR LOOP Statement

- **The cursor FOR LOOP statement implicitly declares its loop index**
- The loop index is %ROWTYPE record variable of the type that the cursor returns
- This **index record is local to the loop and exists only during loop execution.**
- Statements inside the loop can reference the record and its fields
- They can reference virtual columns only by aliases



Cursors FOR LOOP Statement

- After declaring the loop index record variable, the FOR LOOP statement
  - Opens the specified cursor
  - Fetches a row from the result set and stores it in the record with each iteration of the loop
  - Closes the cursor when there are no more rows to fetch

- The cursor also closes if a statement inside the loop
  - Transfers control outside the loop (i.e., using GOTO)
  - PL/SQL raises an exception



Cursors FOR LOOP Statement Example

```
 1 set serveroutput on
 2 DECLARE
 3   CURSOR c1 IS
 4     SELECT region_name rn, country_name cn, country_id cid
 5     FROM regions NATURAL JOIN countries ORDER BY 1, 2;
 6 BEGIN
 7   FOR r1 in c1
 8   LOOP
 9     FOR r2 IN (SELECT city c FROM locations
10                WHERE country_id=r1.cid ORDER BY city)
11     LOOP
12         DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r2.c);
13     END LOOP;
14   END LOOP;
15 END;
16 /
```



Cursor Variables

- A cursor variable is like an explicit cursor, except that:
  - It is **not limited to one query**
  - It can be used to open a cursor variable for a query, process the result set, and then use the cursor variable for another query.
  - **A value can be assigned to it**
  - It can be used in an **expression**
  - It can be **a subprogram parameter**
  - It can be used to **pass query result sets between subprograms**
  - It can be a **host variable**
  - It can be used to **pass query result sets between PL/SQL stored subprograms and their clients**
  - It **cannot accept parameters**
  - Whole queries can be passed to it (the queries can include variables)



Cursor Variables

- A cursor variable offers flexibility because it is a pointer
  - Its value is the address of an item, not the item itself

- Before a cursor variable can be referenced it must point to a SQL work area
  - By opening the SQL work area
  - By assigning it the value of an opened
    - PL/SQL cursor variable
    - Host cursor variable

- Cursor variables and explicit cursors are not interchangeable, one cannot be used where the other is expected
  - E.g., a cursor variable cannot be used in a cursor FOR LOOP statement.



Cursor Variable

- To create a cursor variable
  - Declare a variable of the predefined type SYS_REFCURSOR
  - Define a REF CURSOR type and then declare a variable of that type.
- Informally, a cursor variable is sometimes called a REF CURSOR
- The basic syntax of a REF CURSOR type definition is:
  TYPE type_name IS REF CURSOR [RETURN return_type]
- The REF CURSOR type and cursor variables of that type are strong if **return_type** is specified, otherwise, they are weak.
- SYS_REFCURSOR and cursor variables of that type are weak



Cursor Variable

- Only queries that return the specified type can be associated with a strong cursor variable
- Any query can be associated with a weak cursor variable
- Weak cursor variables are more error-prone than strong ones, but they are also more flexible.
- Weak REF CURSOR types are interchangeable with each other and with the predefined type SYS_REFCURSOR.
- The value of a weak cursor variable can be assigned to any other weak cursor variable
- The value of a strong cursor variable can be assigned to another strong cursor variable only if both cursor variables have the same type (not merely the same return type).



Assigning Values to a Cursor Variable

- PL/SQL cursor variable can be initialized with
  - Another PL/SQL cursor variable
  - A host cursor variable

- The syntax is:

  target_cursor_variable := source_cursor_variable;

- If source_cursor_variable is open, then after the assignment, target_cursor_variable is also open

- The two cursor variables point to the same SQL work area

- If source_cursor_variable is not open, opening target_cursor_variable after the assignment does not open source_cursor_variable



Variables in a Cursor Variable

- The query associated with a cursor variable can reference any variable in its scope
- When a cursor variable is opened with the OPEN FOR statement, PL/SQL
  - Evaluates any variables in the query
  - Uses those values when identifying the result set
- Changing the values of the variables later does not change the result set



Opening a Cursor Variable

- A cursor variable can be opened with the OPEN FOR statement

1. Associates the cursor variable with a query (typically, the query returns multiple rows)
   - The query can include placeholders for bind variables
   - The bind variable values can be specified in the USING clause of the OPEN FOR statement

2. Allocates database resources to process the query

3. Processes the query:
   - Identifies the result set
   - If the query references variables, their values affect the result set
   - If the query has a FOR UPDATE clause, locks the rows of the result set

4. Positions the cursor before the first row of the result set



Fetching Data with a Cursor Variable

- After opening a cursor variable the rows of the query result set can be fetched with the FETCH statement
- The return type of the cursor variable must be compatible with the INTO clause of the FETCH statement
- PL/SQL catches incompatibility
  - At compilation time if the cursor variable is strong
  - At run time if the cursor variable is weak
- The predefined exception ROWTYPE_MISMATCH is raised before the first fetch in case of incompatibility for a weak cursor variable



Closing a Cursor Variable

- A cursor variable needs to be closed before reopening it (that is, using it in another OPEN FOR statement)
- The query previously associated with a cursor variable is lost after it is reopened
- A cursor variable is closed with the CLOSE statement
- Using the CLOSE statement allows the resources associated with the cursor variable to be reused
- After closing a cursor variable
  - Its records cannot be fetched
  - Its result set cannot be accessed
  - Its attributes cannot be referenced
- Trying any of these raises the predefined exception INVALID_CURSOR



Cursor Variable Example

```
DECLARE
  TYPE tbl IS TABLE OF VARCHAR2(50);
  TYPE rc IS REF CURSOR RETURN departments%ROWTYPE;
  c1 rc;
  c2 SYS_REFCURSOR;
  r1 departments%ROWTYPE;
  names tbl;
BEGIN
  OPEN c1 FOR SELECT * FROM departments;
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    OPEN c2 FOR SELECT first_name || ' ' || last_name ename
    FROM employees WHERE department_id=r1.department_id;
    FETCH c2 BULK COLLECT INTO names;
    IF names.COUNT > 0 THEN
      FOR i IN names.FIRST .. names.LAST LOOP
        DBMS_OUTPUT.PUT_LINE(r1.department_name || '::' || names(i));
      END LOOP;
    END IF;
    IF c2%ISOPEN THEN
      CLOSE c2;
    END IF;
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
END;
/
```



Cursor Expressions

- A CURSOR expression returns a nested cursor
- It has this syntax:

  CURSOR (subquery)

- CURSOR expression can be
  - Used in a SELECT statement that is not a subquery
  - Passed to a function that accepts a cursor variable parameter
- A cursor expression cannot be used with an implicit cursor



Cursor Expressions Example

```
DECLARE
  CURSOR c1 IS
    SELECT department_name,
      CURSOR ( SELECT e.last_name || ' ' || e.first_name name
                 FROM employees e
                 WHERE e.department_id = d.department_id
                 ORDER BY e.last_name
               ) employees
    FROM departments d
    ORDER BY department_name;
  c2 SYS_REFCURSOR;
  dname departments.department_name%TYPE;
  ename varchar2(45);
BEGIN
  OPEN c1;
  LOOP  -- Process each row of query result set
    FETCH c1 INTO dname, c2;
    EXIT WHEN c1%NOTFOUND;
    LOOP -- Process each row of subquery result set
      FETCH c2 INTO ename;
      EXIT WHEN c2%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(dname||'::'||ename);
    END LOOP;
  END LOOP;
  CLOSE c1;
END;
/
```



FOR UPDATE

- The SELECT statement with the FOR UPDATE clause (SELECT FOR UPDATE statement) selects the rows of the result set and locks them

- SELECT FOR UPDATE
  - Locks rows for update
  - It ensures that no other user can change those values before an update

- By default, the SELECT FOR UPDATE statement waits until the requested row lock is acquired

- To change this behavior, use the NOWAIT, WAIT, or SKIP LOCKED clause of the SELECT FOR UPDATE statement.



FOR UPDATE

```
DECLARE
  CURSOR c1 IS
    SELECT employee_id, first_name, last_name, salary, department_id
    FROM employees
    WHERE employee_id NOT IN
        (SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT NULL)
    FOR UPDATE OF salary
    ORDER BY department_id;
  asd number(10,2);
  r1 c1%ROWTYPE;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    SELECT avg(salary) INTO asd FROM employees
    WHERE department_id = r1.department_id;
    IF r1.salary > asd THEN
      UPDATE employees SET salary = salary * 1.10
      WHERE
      CURRENT OF c1;
    END IF;
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
  ROLLBACK;
END;
/

select salary from employees;
```



FOR UPDATE

- When SELECT FOR UPDATE is associated with an explicit cursor, the cursor is called a FOR UPDATE cursor
- Only a FOR UPDATE cursor can appear in the CURRENT OF clause of an UPDATE or DELETE statement.
- The CURRENT OF clause
  - Is a PL/SQL extension to the WHERE clause of the SQL statements UPDATE and DELETE
  - Restricts the statement to the current row of the cursor
- When SELECT FOR UPDATE queries multiple tables, it locks only rows whose columns appear in the FOR UPDATE clause.



Bibliography

- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 19c*, Oracle Corporation, 2022 [pdf]
- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 21c*, Oracle Corporation, 2022 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 19c*, Oracle Corporation, 2020 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 21c*, Oracle Corporation, 2021 [pdf]